MLOPS COURSE PROJECT · 2026

Loan Default Prediction Using Machine Learning

An end-to-end MLOps pipeline: from raw loan data to a monitored, self-retraining prediction service.

5,960
loan records

4
models compared

10
MLOps tools integrated

0.95
final ROC-AUC

Presented by [Aditya B Kale] · [Lavesh V Hasani] · [Yash S Khobragade] | []

Current problem with detecting loan default

Manual checking miss some features

Manual checking is done by officers and they read the application and decide whether a person is suitable for giving loan or not , This can miss some features as each officer judges differently Officers only use simple if then features and not variety of features Good customers can be rejected many times

Manual review is slow

Manual review is slow and bothersome , inacurrate due to missing features and thus cant be scaled

No learning from past dara

Banks have thousands of records but the manual checks dosent learnt

THE BUSINESS QUESTION

"Can we predict whether a borrower will default *before* the loan is approved?"

Our goal: a model that flags high-risk applicants early, so the bank cuts losses and approves good customers faster.

The HMEQ dataset at a glance

5,960
home-equity loan applications

12 + 1
input features plus the target BAD

19.9%
defaulted (1,189 loans)

Group	Features	What they describe
Loan & property	LOAN, MORTDUE, VALUE	Loan, mortgage owed, home value
Applicant	REASON, JOB, YOJ	Loan purpose, job, years in job
Credit history	DEROG, DELINQ, CLAGE	Bad marks, late lines, credit age
Credit activity	NINQ, CLNO	Inquiries, number of credit lines
Affordability	DEBTINC	Debt-to-income ratio (%)
Target	BAD	1 = defaulted, 0 = repaid

Imbalanced target: 4,771 repaid (80.1%) vs 1,189 defaulted (19.9%)

EDA findings and preprocessing

WHAT THE DATA TOLD US

Missing data: 11 of 12 features have gaps; DEBTINC is missing for 21.3% of loans.

Missing is a signal: loans with no DEBTINC default about 3× more often.

Strongest signals: delinquencies, derogatory reports, high DEBTINC, short credit history.

Imbalance: only 1 in 5 loans defaults, so accuracy alone is misleading.

WHAT WE DID ABOUT IT

1. Median / mode imputation
   Missing numeric columns – like DEBTINC -took the median

   Missing text Coloum – like jobs – took the mode

2. One-hot encoding for REASON and JOB

   Converting everything to 1/0 format

3. Standard scaling of numeric features
   loans are larger numbers , scaling puts everything on same scale

4. Stratified 80/20 split: 4,768 / 1,192 loans

Training four candidate models

01 · BASELINE

Logistic Regression

Simple, fast and explainable; shows how far a linear boundary can go.
Tuned: C, L2 penalty

02 · RULES

Decision Tree

Learns if-then rules like a credit officer would, but overfits easily.
Tuned: max_depth, min_samples_leaf

03 · ENSEMBLE

Bagging Classifier

Averages many trees trained on bootstrapped samples to cut variance.
Tuned: n_estimators, max_samples

04 · ENSEMBLE+

Random Forest

Bagging plus random feature subsets, so trees are less correlated and more robust.
Tuned: n_estimators, max_depth, max_features

✓ 5-fold stratified cross-validation for every model

Grid-search tuning: 48 runs in total

Optimised for recall on the default class

Experiment tracking with MLflow

Parameters
Hyperparameters, preprocessing options, random seed, DVC data version.

Metrics
Accuracy, precision, recall, F1 and ROC-AUC per fold and on the test set.

Artifacts
Trained pipeline file, confusion matrix, ROC curve, feature importances.

Model registry
Best run registered as loan-default-rf, moving Staging → Production.

TOP 5 OF 48 RUNS, BY RECALL

Run	Model	Recall	ROC-AUC
rf-tuned-07	Random Forest	0.79	0.95
rf-baseline	Random Forest	0.74	0.94
bag-tuned-03	Bagging	0.72	0.93
dt-tuned-05	Decision Tree	0.68	0.79
lr-tuned-02	Logistic Reg.	0.58	0.81

rf-tuned-07 → registered as loan-default-rf v3, stage: Production

> Every run links to its Git commit and DVC data version, so any result can be reproduced exactly.

Comparing models on the test set

Model	Accuracy	Precision	Recall (key)	F1-score	ROC-AUC
Logistic Regression	0.85	0.63	0.58	0.60	0.81
Decision Tree	0.87	0.66	0.68	0.67	0.79
Bagging Classifier	0.91	0.80	0.72	0.76	0.93
Random Forest (selected)	0.93	0.84	0.79	0.81	0.95

0.79

Highest recall

Flags 188 of the 238 real defaulters in the test set.

0.95

Best ROC-AUC

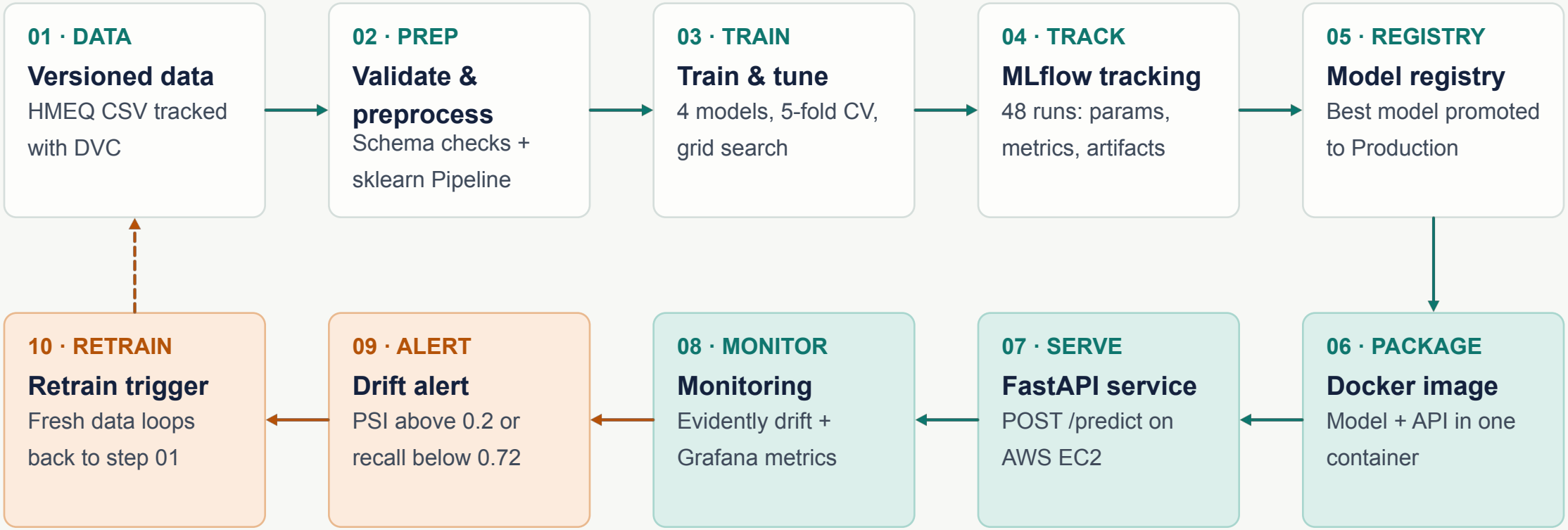
Ranks risky borrowers above safe ones most reliably.

±0.02

Most stable

Recall barely changes across CV folds; feature importances aid explanation.

End-to-end MLOps pipeline



GitHub Actions automates every arrow: each push re-tests, retrains and redeploys the pipeline.

CI/CD with GitHub Actions

CONTINUOUS INTEGRATION

On every push and pull request

1
Lint & format
flake8 and black on every file

2
Unit tests
34 pytest tests for preprocessing and the API

3
Data validation
Schema, type and value-range checks

CONTINUOUS DELIVERY

On merge to the main branch

4
Retrain
dvc repro reruns only changed stages

5
Promotion gate
New model must match or beat production

6
Build & push
Docker image to the registry

7
Deploy
Rolling update on the EC2 server

Promotion gate: deploy only if recall ≥ production recall and ROC-AUC drops by at most 0.01. Full run takes about 9 minutes.

Serving predictions with FastAPI

EXAMPLE REQUEST

```
POST /predict
{"LOAN": 18000, "MORTDUE": 62000, "VALUE": 85000,
 "REASON": "DebtCon", "JOB": "Other", "YOJ": 2,
 "DEROG": 1, "DELINQ": 3, "CLAGE": 96, "NINQ": 4,
 "CLNO": 21, "DEBTINC": 44.5}
```

RESPONSE

```
{"default_probability": 0.72,
 "risk_label": "High",
 "model_version": "loan-default-rf v3"}
```

42 ms p95 latency
Fast enough for real-time loan screening.

Docker image, 410 MB
python:3.11-slim base; runs the same everywhere.

AWS EC2 t3.medium
A /health endpoint supports uptime checks.

Risk bands
Low: below 0.30 · Medium: 0.30–0.60 · High: above 0.60

Monitoring and drift detection

Signal	Tool & frequency	Alert threshold	Action
Data drift	Evidently, weekly	PSI above 0.2 on a top-5 feature	Alert + auto-retrain
Prediction drift	Evidently, weekly	High-risk share shifts over 10 pts	Investigate pipeline
Model quality	Repayment labels, monthly	Recall below 0.72	Auto-retrain via CI/CD
Service health	Prometheus + Grafana	p95 over 200 ms or errors over 1%	Alert the on-call member

DRIFT SIMULATION

Synthetic applicants with higher DEBTINC

3 of 12
features flagged as drifted

0.70
recall after drift (below 0.72)

0.78
recall after auto-retrain

PSI = Population Stability Index

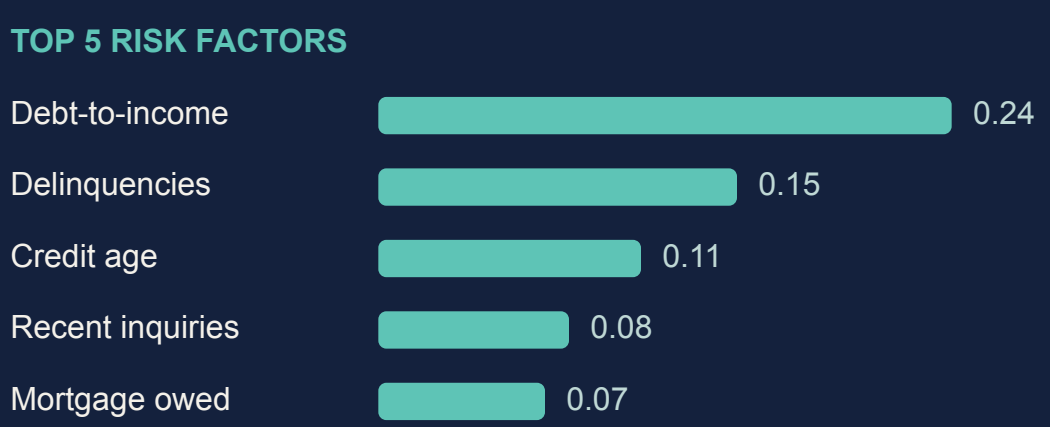
Results and business impact

0.79
Recall: 4 in 5 defaulters caught

0.95
ROC-AUC

93%
Accuracy

0.81
F1-score (precision 0.84)



Business impact: the logistic baseline let 100 defaulters through the test set; Random Forest lets only 50 through, half as many bad loans approved.

Random Forest on 1,192 unseen test loans

Speaker 3 · [Name 3]

Conclusion and future scope

Key takeaways

- Random Forest reached 0.79 recall and 0.95 ROC-AUC on unseen loans.
- DVC, MLflow, Docker and monitoring make results reproducible and reliable.
- Automation turns a notebook into a self-maintaining service.

Limitations

- HMEQ is small (5,960 loans) and from one period and region.
- Heavy missing data means imputation may hide real patterns.
- No fairness audit yet across applicant groups.

Future scope

- Try XGBoost and LightGBM with probability calibration.
- Add SHAP explanations for every loan decision.
- Fairness checks, A/B testing and AWS SageMaker deployment.

Thank you!

Questions? We're happy to discuss any part of the pipeline.